

Module 5 Question Pool

OMSCS 6250 Computer Networks

Lesson 5: Router Design and Algorithms (Part 1)

What's Inside a Router?

Q1. [MCQ]

A router's functionalities are divided between two planes: the data plane and the control plane.

Which functionalities is each plane responsible for?

- **A.** The data plane runs the routing protocols; the control plane moves each packet from its input link to its output link.
- **B.** The data plane moves each packet to the correct output link; the control plane computes the forwarding table.
- **C.** The data plane forwards the data packets sent by the hosts; the control plane forwards the BGP packets sent by the routers.
- **D.** The data plane queues packets at the output ports; the control plane selects which queued packet is sent next.

Correct answer: **B**

Why: The data plane forwards packets. It moves each arriving packet to the right output link. The control plane runs the routing protocols, such as OSPF and BGP, and computes the forwarding table. Queuing packets and choosing the next one to send are data plane work. The control plane does not forward BGP messages. It reads the BGP messages sent to the router. The data plane forwards every packet that passes through the router, whoever sent it.

Q2. [MCQ]

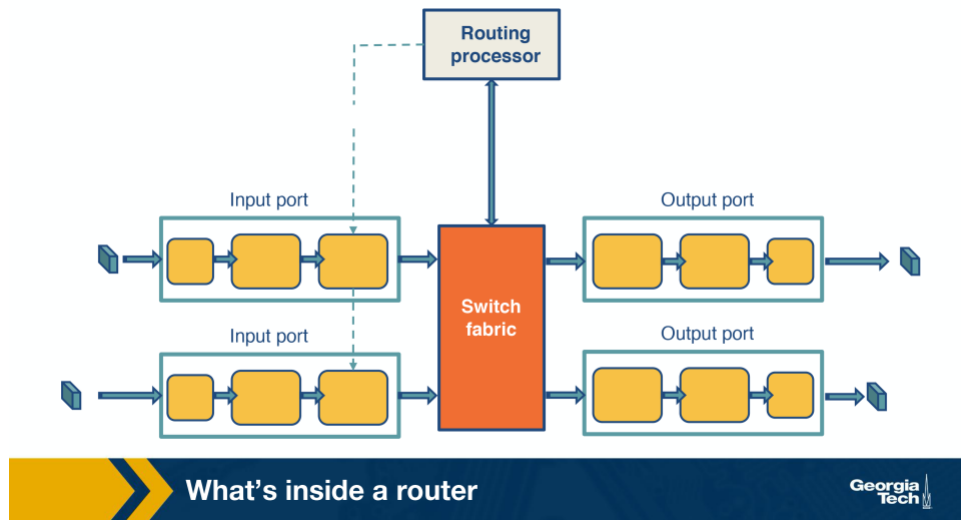


Figure: Inside a router (Module 5)

Consider a high-speed router. The router has a routing processor, several line cards and a switching fabric. Each line card has input ports and output ports. The switching fabric connects the input ports to the output ports.

Which parts of the router run the control plane, and which parts run the data plane?

- **A.** The routing processor runs the control plane; the line cards and the fabric run the data plane.
- **B.** The line cards run the control plane, since each holds a table; the processor runs the data plane.
- **C.** The routing processor runs both planes: it looks up each packet and then sets up the fabric.
- **D.** The line cards run both planes: each line card computes its own table and forwards packets.

Correct answer: **A**

Why: The routing processor runs the control plane in software. It runs the routing protocols and computes the forwarding table. The line cards and the fabric run the data plane in hardware. Each line card looks up each packet in its own copy of the table, and the fabric carries the packet to its output port. A line card holds a copy of the table but does not compute it. Sending every packet to the routing processor would overload it.

Q3. [TF]

A high-speed router has a routing processor and several line cards. The routing processor runs the routing protocols and computes the forwarding table.

For each arriving data packet, the input port asks the routing processor which output port to use.

- True
- False

Correct answer: **False**

Why: Each line card holds its own copy of the forwarding table. So the input port finds the output port by itself. The routing processor is not asked about individual data packets.

Q4. [TF]

Two packets arrive at a router on two different input ports. The switching fabric carries both packets to the same output port. The outgoing link of that output port is busy sending a third packet. The output port places the two packets in its queue.

The output port decides which of the queued packets goes out on the link next.

- True
- False

Correct answer: **True**

Why: Choosing which queued packet goes out next is called output scheduling. The output port does it. It can use first-come first-served, or weighted fair queuing.

Q5. [MCQ]

A link attached to a router fails. The router then performs four operations, in this order.

1. The OSPF process on the router builds a new link-state advertisement and sends it to the neighbors of the router. 2. The OSPF process computes new shortest paths to every destination. 3. The routing processor installs the new forwarding table on each line card. 4. An input port looks up the destination of the next arriving packet in the new forwarding table.

Which operations are control plane operations?

- A. Operations 1 and 2.
- B. Operations 2 and 3.
- C. Operations 1, 2 and 3.
- D. Operations 1, 2, 3 and 4.

Correct answer: **C**

Why: Operations 1, 2 and 3 are control plane work. The OSPF process builds and sends the link-state advertisement and computes the new paths. The routing processor then installs the new forwarding table on each line card. The output ports only carry the advertisement out. Operation 4 is data plane work. The input port uses the table to forward a packet.

Q6. [MCQ]

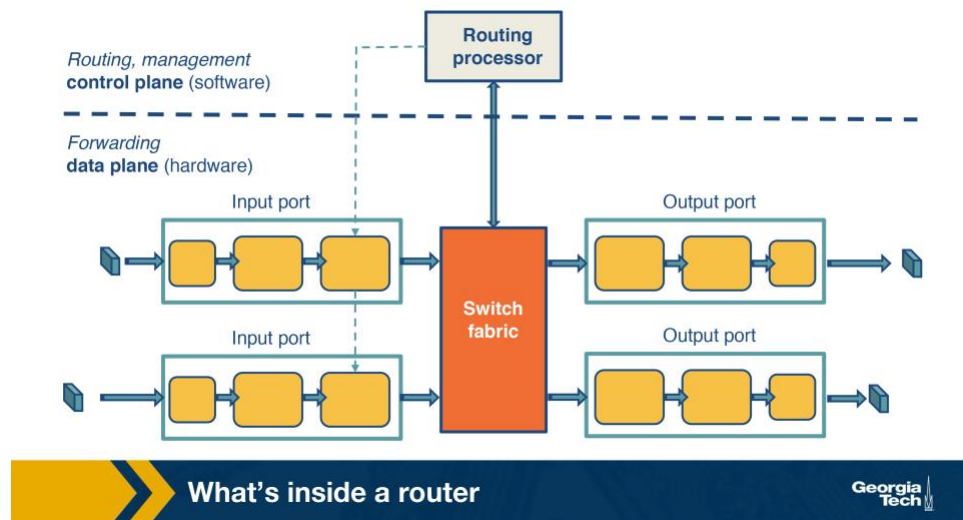


Figure: Inside a router, control plane and data plane (Module 5)

Consider a high-speed router. Each line card of the router holds a copy of the forwarding table. The router has no backup routes. The routing processor of the router crashes, and it takes 30 seconds to restart. The line cards and the switching fabric keep running. The neighbors of the router do not notice the restart. Ten seconds after the crash, one of the output links of the router fails.

What happens to the packets that arrive at the router while the routing processor restarts?

- **A.** The line cards forward them with the old table copies; packets sent to the failed link are lost.
- **B.** The line cards drop them, because no routing processor is running to choose their output ports.
- **C.** The line cards keep them in the input queues until the routing processor restarts and updates the table.
- **D.** The line cards forward them with the old table copies and compute a new table when the link fails.

Correct answer: **A**

Why: Each line card forwards packets with its own copy of the forwarding table. So forwarding goes on while the routing processor restarts. Packets are not dropped or held back just because the processor is down. After the link fails, only the routing processor can compute a new table. The line cards cannot. So the copies are not updated until the restart ends, and packets sent to the failed link are lost.

Q7. [MCQ]

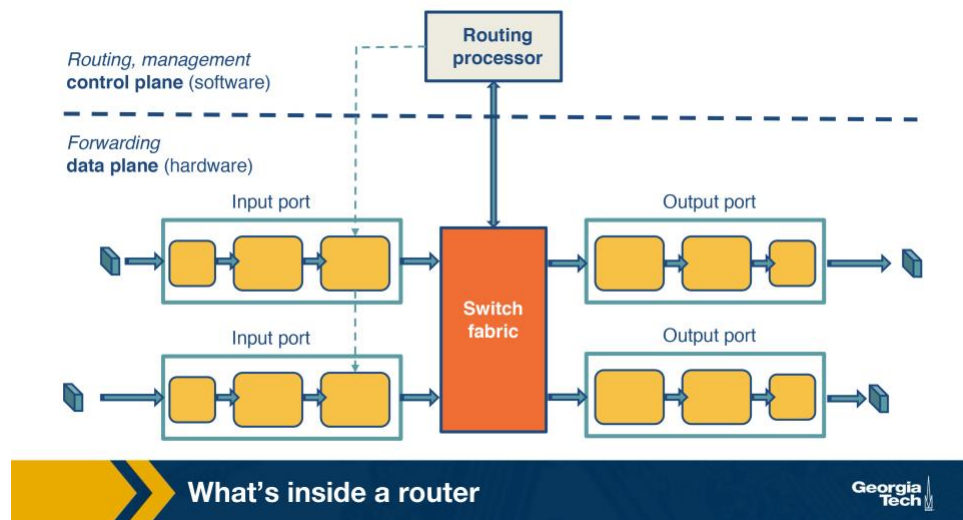


Figure: Inside a router, control plane and data plane (Module 5)

Consider a high-speed router that runs OSPF with its neighbors. Each line card of the router has input ports and output ports. A network operator powers off all the line cards of the router for maintenance. The routing processor stays up.

How does powering off the line cards affect the control plane of the router?

- **A.** The control plane keeps working, because the routing processor is separate from the data plane.
- **B.** The routing processor can no longer exchange OSPF messages, since they pass through the ports.
- **C.** The routing processor stops running OSPF, because the OSPF process runs on the line cards.
- **D.** The routing processor keeps its routes current, since OSPF messages reach it over a separate bus.

Correct answer: **B**

Why: OSPF messages from a neighbor arrive at an input port, which passes them to the routing processor. Messages from the routing processor leave through an output port. With the line cards off, the router has no working ports, so the routing processor cannot exchange messages with its neighbors. OSPF runs on the routing processor and keeps running. The separate bus connects the routing processor to the line cards. With the line cards off, the bus has nothing to connect to.

Q8. [MCQ]

An operator runs OSPF on all the routers in its network. The operator plans to migrate the control plane functions of every router to a remote controller.

After the migration, which operation does each router still perform itself?

- **A.** The router runs OSPF with its neighbors to keep its own link-state database current.
- **B.** The router computes its own forwarding table, and the controller only collects copies of it.
- **C.** The router asks the controller for the output port of each packet that arrives.
- **D.** The router looks up each arriving packet in its local forwarding table and forwards it.

Correct answer: **D**

Why: Forwarding is data plane work, so it stays at the router. Each router looks up each arriving packet in its local table. Computing routes and forwarding tables is control plane work, so it moves to the controller. The routers stop running OSPF. The controller computes the table of each router and installs it there. The router forwards each packet itself. Asking the controller about each packet would add delay to every packet.

Router Architecture

Q9. [MCQ]

A router performs the following five tasks.

1. The router looks up the destination address of each packet in its forwarding table. 2. The router sends an ICMP error message to the source when the TTL of a packet has run out. 3. The switching fabric carries each packet from its input port to its output port. 4. The router runs OSPF to build its routing table. 5. The output port decides which queued packet goes out on the link next.

Which three tasks add to the delay of every packet that crosses the router?

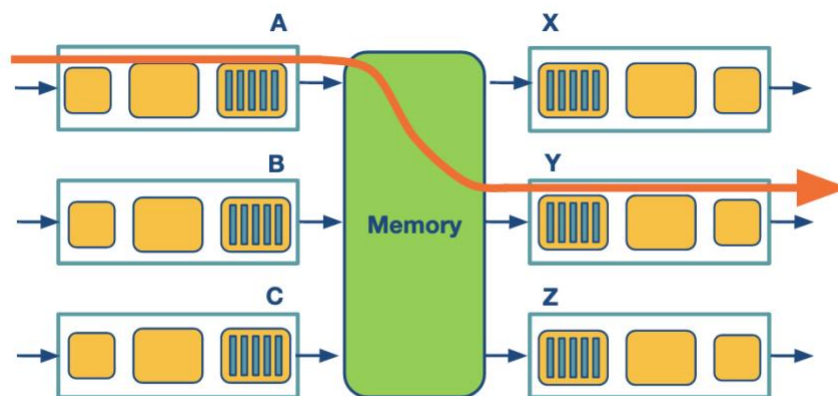
- A. Tasks 1, 2 and 3.
- B. Tasks 1, 3 and 5.
- C. Tasks 1, 3 and 4.
- D. Tasks 2, 3 and 5.

Correct answer: **B**

Why: Tasks 1, 3 and 5 happen for every packet: the lookup, the trip across the fabric, and output scheduling. Each one adds to the delay of every packet. Task 2 happens only for the rare packet whose TTL has run out. Task 4, OSPF, builds the routing table in the background. It does not handle each packet.

Different Types of Switching

Q10. [MCQ]



Switching via memory

Figure: Switching via memory (Module 5)

Consider an early router that switches packets via memory. The input ports and output ports work as I/O devices controlled by the routing processor. The memory of the routing processor supports at most B packet reads or packet writes per second.

What limit does the memory place on the forwarding throughput of the router?

- A. It stays below $B/2$ packets per second, since each packet is written in and then read out.
- B. It can reach B packets per second, since each packet needs one memory operation to cross.
- C. It can reach $2B$ packets per second, since one packet's write can overlap another's read.
- D. It stays below B/N packets per second for N ports, since the ports take turns on the memory.

Correct answer: A

Why: Each packet is written into memory and then read out. That is two memory operations per packet. The memory does one operation at a time, so the router moves fewer than $B/2$ packets per second. A write cannot overlap a read. The number of ports does not change the limit.

Q11. [MCQ]

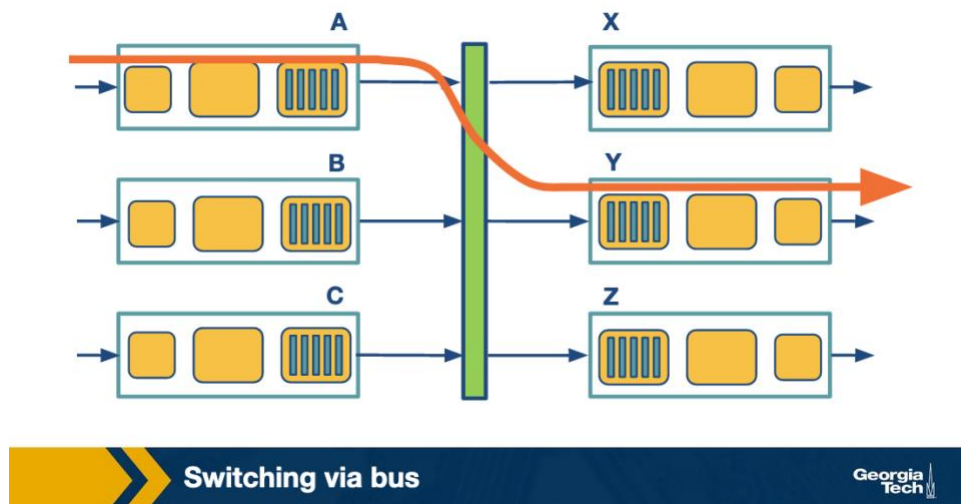


Figure: Switching via shared bus (Module 5)

Consider a router that switches packets via a bus. When an input port receives a packet, the input port adds an internal header that designates the output port. The input port then sends the packet onto the bus. Packets arrive at several input ports at the same time.

What limits the forwarding rate of the router?

- A. The routing processor, since each packet raises an interrupt and is copied through its memory.
- B. The bus, since it carries one packet at a time and its speed caps the speed of the router.
- C. The output ports, since each output port receives the packet and has to process it.
- D. The internal header, since the designated output port must strip it from every packet.

Correct answer: **B**

Why: The bus carries one packet at a time, so the speed of the bus limits the router. The routing processor is not on the path. Every output port sees the packet, but only the chosen port keeps it and removes the internal header. That small step does not limit the rate.

Q12. [MCQ]

A router uses a crossbar switch to connect N input ports to N output ports. The switching fabric closes a crosspoint to connect an input port to an output port.

How many buses does the crossbar switch need?

- **A.** N^2 buses: one bus for each crosspoint, so that each input can reach each output at once.
- **B.** N buses: each port's bus carries its input traffic in and its output traffic back out.
- **C.** $2N$ buses: N horizontal input buses and N vertical output buses that meet at crosspoints.
- **D.** $N + 1$ buses: one horizontal bus per input, plus one shared bus that the outputs listen to.

Correct answer: **C**

Why: Each input port has a horizontal bus, and each output port has a vertical bus. That makes $2N$ buses. They cross at N^2 crosspoints, and a crosspoint is not a bus. N buses are too few, because inputs and outputs need separate buses. No output bus is shared, so the count is not $N + 1$.

Q13. [MCQ]

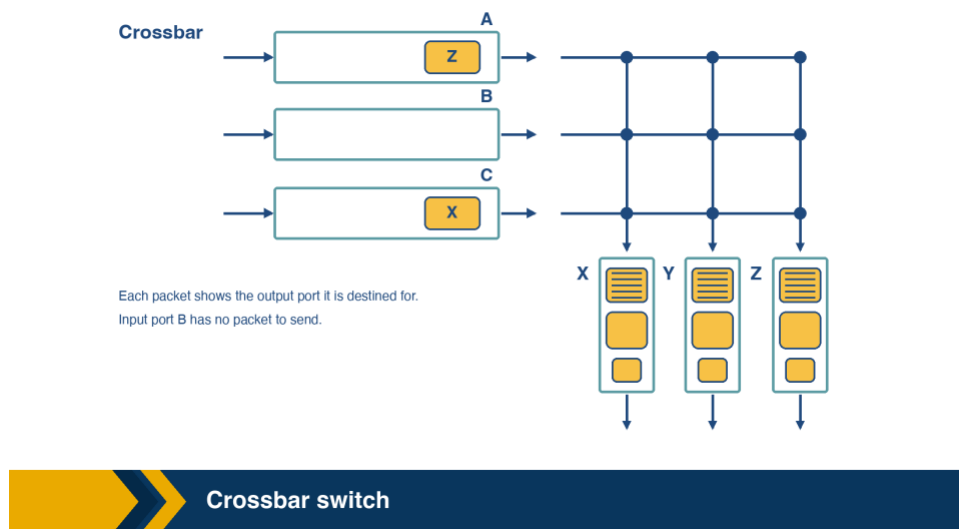


Figure: Crossbar switch (Module 5)

Consider a router that uses a crossbar switch with input ports A, B and C and output ports X, Y and Z. No other transfers are in progress. Input port A has a packet for output port Z. Input port C has a packet for output port X.

Can the two packets cross the switching fabric at the same time?

- **A.** No, because a crossbar carries one packet at a time when several inputs are active.
- **B.** No, because the A-to-Z path crosses the C-to-X path, and the crossing blocks one of the transfers.
- **C.** Yes, provided the fabric runs twice as fast as the links, since two packets cross it.
- **D.** Yes, because the two transfers use different input buses and different output buses.

Correct answer: **D**

Why: The fabric closes two crosspoints, A to Z and C to X. The two transfers use different buses, so both packets cross at once. Buses that cross do not block each other unless their crosspoint is closed. The fabric does not need to run faster.

The Challenges Routers Face

Q14. [MCQ]

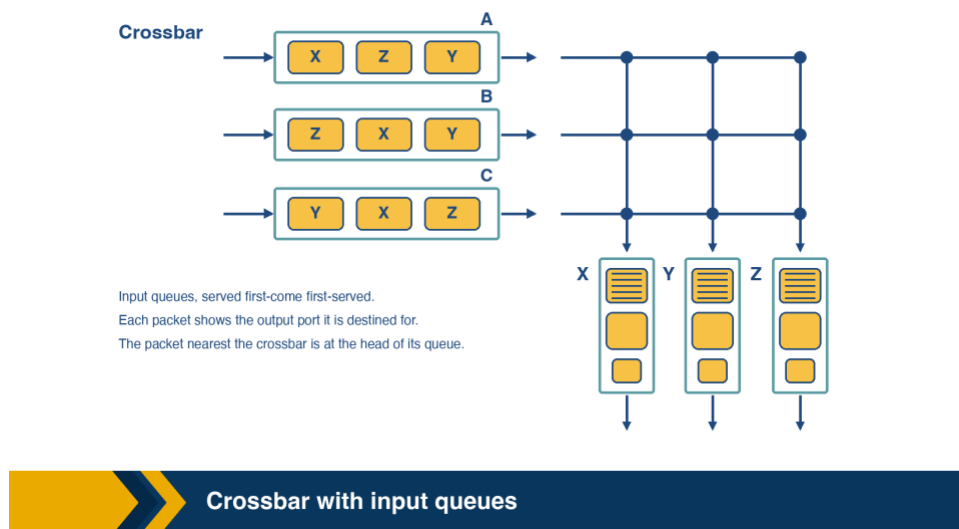


Figure: Crossbar with input queues (Module 5)

Consider a router that uses a crossbar switch with input ports A, B and C and output ports X, Y and Z. Packets wait in input queues and are served first-come first-served. The packet at the head of the input queue of port A is destined for output port Y. The packet at the head of the input queue of port B is also destined for output port Y. Other packets wait behind the head of each input queue.

What happens when both packets try to cross the switching fabric at the same time?

- **A.** One transfer goes ahead; the other packet waits in its input queue and blocks packets behind it.
- **B.** Both go ahead, since the output bus interleaves the bits of the two packets in the same interval.
- **C.** The fabric sends one packet to a free output port instead, and that port forwards it onward.
- **D.** The fabric drops one of the two packets, since a crossbar has no place to hold the one that loses.

Correct answer: **A**

Why: Output Y takes one packet at a time, so one transfer waits. The waiting packet stays at the head of its input queue. The packets behind it wait too, even if their outputs are free. This is head-of-line blocking. The fabric does not mix the bits of two packets, send a packet to another output, or drop the waiting packet.

Q15. [MCQ]

A router uses service differentiation. It gives the traffic sent by different customers different quality-of-service guarantees.

What additional per-packet work does service differentiation require from the router at high speed?

- **A.** Sending each packet to the routing processor, which looks up the service guarantee that its customer bought.
- **B.** Keeping a separate forwarding table for each service guarantee on each line card, and choosing among them.
- **C.** Classifying each packet by fields other than its destination, such as its source or application.
- **D.** Looking up each packet's destination a second time, in a table that maps destinations to service guarantees.

Correct answer: **C**

Why: Service differentiation treats packets differently based on more than their destination. So the router must sort each packet by other header fields, such as its source or its application. This is called packet classification. The routing processor is not asked about each packet. A second destination lookup still sorts packets only by destination. Separate forwarding tables only change the output port, not the service a packet gets.

Prefix-Match Lookups

Q16. [MCQ]

Before 1993, IPv4 used classful addressing. A network prefix could be only 8, 16 or 24 bits long. In 1993, CIDR replaced classful addressing.

Why does a router that uses CIDR need longest prefix matching instead of exact matching?

- **A.** Each address matches only one prefix, but the router may have to try all 32 lengths to find it.
- **B.** CIDR made the prefixes shorter, so an exact match on the full 32-bit address now fails.
- **C.** The address no longer shows the prefix length, and one address can match several prefixes.
- **D.** Longest prefix matching needs fewer memory accesses per lookup than an exact match does.

Correct answer: **C**

Why: Under classful addressing, the first bits of an address showed the prefix length, so an exact match worked. With CIDR, a prefix can have any length, and the address does not show it. One address can match several prefixes, such as a large block and a smaller block inside it, and the router picks the longest. Exact matching used only the network part of the address. CIDR allows both shorter and longer prefixes. Longest prefix matching takes more work.

Q17. [MCQ]

The forwarding table of a router holds two entries:

10.0.0.0/8, next hop provider P

10.1.2.0/24, next hop provider Q

A packet with destination address 10.1.2.7 arrives at the router.

How does the router handle this packet?

- **A.** It forwards the packet to provider P via the 10.0.0.0/8 entry.
- **B.** It forwards the packet to provider Q via the 10.1.2.0/24 entry.
- **C.** It drops the packet, since no entry matches the address exactly.
- **D.** It splits the traffic, sending packets in turn to P and to Q.

Correct answer: **B**

Why: Both prefixes match 10.1.2.7. The router picks the longer one, the /24, so the packet goes to provider Q. No exact match is needed. The router does not split traffic between the two entries.

Q18. [MCQ]

A router reaches the address block 10.0.0.0/8 through provider P. One customer inside that block uses the addresses 10.1.2.0/24. That customer moved to a different provider Q and kept its addresses. The forwarding table of the router now holds two entries:

10.0.0.0/8, next hop provider P

10.1.2.0/24, next hop provider Q

A packet with the destination address 10.1.2.7 arrives at the router. The address 10.1.2.7 falls inside both prefixes. The router forwards the packet to provider Q.

Why does the router use the 10.1.2.0/24 entry rather than the 10.0.0.0/8 entry?

- **A.** The /24 entry was added to the table more recently than the /8 entry.
- **B.** The /24 entry offers a shorter path to 10.1.2.7 than the /8 entry.
- **C.** The /24 entry has a lower-cost path to 10.1.2.7 than the /8 entry.
- **D.** The /24 prefix is the longest of the prefixes that match 10.1.2.7.

Correct answer: **D**

Why: Both prefixes match 10.1.2.7. The router uses the longest matching prefix, the /24. How new an entry is does not matter. The length or cost of a path does not matter either. Those are routing metrics, which the control plane uses to build the table. The lookup compares only prefix lengths.

Q19. [TF]

A backbone router carries about 250,000 concurrent flows of short duration. The router keeps a cache of recently seen destination addresses.

The cache lets the router skip most lookups in the forwarding table.

- True
- False

Correct answer: **False**

Why: With about 250,000 short flows at once, a recently seen destination is rarely seen again while it is still in the cache. The cache finds few addresses, and most packets still need a full lookup.

Unibit Tries

Q20. [MCQ]

A unibit trie stores a database of IP prefixes. A search reads the destination address one bit at a time, starting at the root.

What is the largest number of child pointers that a node of the trie can hold?

- A. One pointer, which leads to the next node on the search path.
- B. Two pointers, a 0-pointer and a 1-pointer, one per bit value.
- C. Three pointers: a 0-pointer, a 1-pointer and a prefix pointer.
- D. A number that grows with the count of prefixes below the node.

Correct answer: **B**

Why: Each node has a 0-pointer and a 1-pointer, one per bit value. Either one may be empty. A stored prefix sits inside the node, not behind a third pointer. The number of prefixes below a node does not change its number of pointers.

Q21. [MCQ]

A unibit trie stores a database of IP prefixes. A search for a destination address starts at the root and moves down the trie one node at a time.

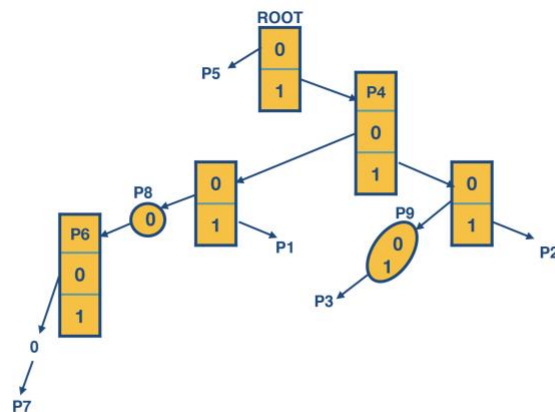
At each node, what decides which child pointer the search follows?

- A. The prefix stored at the node, after the search compares it with the address bits.
- B. The skip count stored at the node, which says how many address bits to jump over.
- C. The side that leads to a longer prefix, after the search looks down both sides.
- D. The next bit of the address: a 0 takes the 0-pointer, and a 1 takes the 1-pointer.

Correct answer: **D**

Why: The next bit of the address picks the pointer: a 0 takes the 0-pointer, and a 1 takes the 1-pointer. The prefix stored at a node is only remembered as the best match so far. It does not steer the search. A skip count, used in Patricia tries, says how many bits to skip, not which way to go. The search never looks down both sides. It follows one path.

Q22. [MCQ]



Prefix-Match Lookups: Unibit Tries Georgia Tech

Figure: Unibit trie: compressed branch P9 (Module 5)

Consider a unibit trie that stores the prefixes P1 to P9. The prefix P3 = 11001* is 5 bits long. No other prefix in the database begins with the same 4 bits as P3. The trie could store the last 2 bits of P3 as two nodes, each with only one pointer. Instead, the trie stores the last 2 bits of P3 as the text string 01, in the node that holds P9.

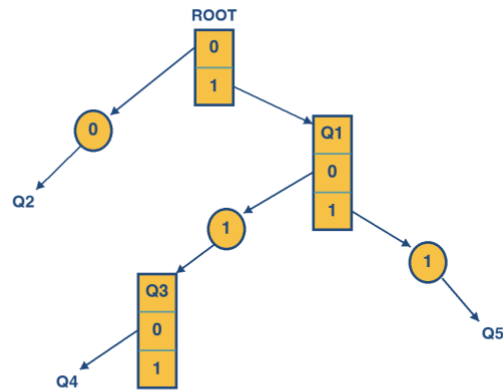
Why does the trie store the last 2 bits of P3 as a text string?

- A. It cuts the search from 32 memory accesses to a few, which fits the lookup budget.
- B. Each bit with no branching would otherwise take a whole node, and that wastes memory.
- C. Without the string, the search could reach P3 by two paths and return the wrong prefix.
- D. The string lets the search skip the 2 bits without comparing them with the address.

Correct answer: **B**

Why: Without the string, each of the last 2 bits of P3 would need a whole node, with one empty pointer. That wastes memory. The string 01 stores the same 2 bits in less space. The search still compares the 2 bits with the address, and there is still only one path to P3. In a dense trie, compression barely shortens the search, so it does not fix the lookup budget.

Q23. [MCQ]



Unibit trie

Figure: Unibit trie (Module 5)

Consider a unibit trie that stores the following five prefixes:

Q1 = 1*

Q2 = 00*

Q3 = 101*

Q4 = 1010*

Q5 = 111*

A packet arrives with a destination address that begins with 10110.

Which prefix does the search return for this address?

- A. Q1 = 1*, the first prefix the search passed before it found an empty pointer.
- B. Q3 = 101*, the last prefix the search passed before it found an empty pointer.
- C. Q4 = 1010*, the last prefix the search passed before it found an empty pointer.
- D. No prefix, since the search failed before it reached the last bit of the address.

Correct answer: **B**

Why: The search follows the bits 1, 0 and 1. It passes Q1 and then Q3, and it remembers each one. The fourth bit is 1, but the node that holds Q3 has no 1-pointer. So the search stops and returns Q3, the last prefix it passed. Q3 is longer than Q1. Q4 = 1010 does not match, because the fourth bit of the address is 1.*

Q24. [MCQ]

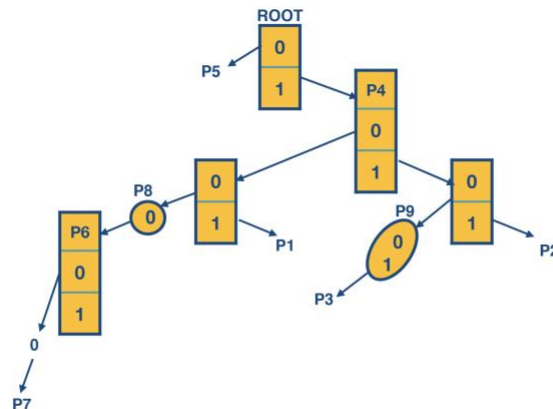
How does compression affect the average height of the trie, and why?

- A. It cuts the height by about half, because each compressed node replaces a whole chain of nodes.
- B. It lowers the height only a little, because few nodes along a lookup path have just one child each.
- C. It lowers the height only a little, because backbone prefixes are short, so the trie is already shallow.
- D. It leaves the height unchanged, because compression saves memory but does not shorten any lookup path.

Correct answer: **B**

Why: Compression removes only chains of nodes with one child. A backbone table is densely populated, so few nodes along a lookup path have just one child. So there are few chains to remove, and the height drops only a little, far from half. The small gain comes from the lack of chains, not from short prefixes: the trie is not shallow. Each chain that is compressed still shortens the paths through it, so the height does drop.

Q26. [MCQ]



Prefix-Match Lookups: Unibit Tries

Georgia Tech

Figure: Unibit trie (Module 5)

Consider a unibit trie for the prefix database P1 to P9. A destination address begins with the bits 11001. The search starts at the root of the trie.

Which prefix does the search return?

- A. P9, because the search passes P4, reaches P9, and the 01 string does not match.
- B. P4, because 1* is the first prefix the search matches, and the first match is returned.
- C. P3, because the search passes P4 and P9, matches the 01 string, and then reaches P3.
- D. P2, because the second bit is 1, and the 11 branch of the trie ends at the prefix 111*.

Correct answer: **C**

Why: The bits 1, 1 and 0 lead past P4 to the node that holds P9. The string 01 in that node matches the next two bits, so the search reaches P3. The search returns the last prefix it passes, which is the longest match. So it returns P3, not P4 or P9. The branch toward P2 = 111* is left behind at the third bit.

Multibit Tries

Q27. [MCQ]

A multibit trie uses a stride of 8. Each step of a search reads 8 bits of a 32-bit IPv4 address.

How many entries does each node of the trie hold?

- A. 8 entries, one for each bit that the node reads.
- B. 16 entries, two for each bit that the node reads.
- C. 4 entries, one for each 8-bit step in a 32-bit address.
- D. 256 entries, one for each value the 8 bits can take.

Correct answer: **D**

Why: Each node reads the next 8 bits of the address. Those 8 bits can form $2^8 = 256$ different patterns, from 00000000 to 11111111. The node holds one entry for each pattern, so it has 256 entries. Some entries may be empty, because no prefix continues through that pattern. So a node always has 256 entries, but it can have fewer than 256 children.

Q28. [MCQ]

A multibit trie uses a stride of 8. Each step of a search reads 8 bits of a 32-bit IPv4 address.

What is the worst-case number of memory accesses for a lookup?

- A. 4, one memory access for each 8-bit step of the address.
- B. 5, four steps plus one more access to read the prefix.
- C. 8, two per step: one for the pointer and one for the prefix.
- D. 32, one memory access for each bit of the address.

Correct answer: **A**

Why: Each step reads 8 bits of the address in one memory access. A 32-bit address has four groups of 8 bits, so a search takes at most 4 accesses. Each access brings back the pointer and the stored prefix together, so no step needs an extra access.

Q29. [TF]

Consider a router that stores its forwarding table in a unibit trie and looks up 32-bit IPv4 addresses. A lookup reads the trie one node at a time, and reading one node takes one memory access of 50 nsec. Assume that the router has a budget of 300 nsec for each address lookup, so that it keeps up with packets arriving back to back.

The worst-case lookup finishes within the 300 nsec budget.

- True
- False

Correct answer: **False**

Why: A unibit trie reads one bit at each node, so the worst case visits 32 nodes. That is 32 accesses of 50 nsec each, or 1,600 nsec (more than five times the budget).

Prefix Expansion

Q30. [MCQ]

A fixed-stride trie reads k bits of the destination address at each step of a search. The prefix database holds prefixes of many different lengths.

Which prefix lengths can the trie store directly, without changing the prefix?

- A. Any length up to 32, since a step can stop partway through its k bits.
- B. Only lengths shorter than k , since each fits inside a single step.
- C. Only lengths that are multiples of k , such as k , $2k$ and $3k$ bits.
- D. Only the length k itself, the number of bits every node reads.

Correct answer: **C**

Why: Each step reads exactly k bits, so a search can stop only after k , $2k$, $3k$ bits, and so on. Only prefixes of those lengths fit. A step cannot stop partway. A prefix shorter than k ends inside the first step, so it does not fit either. The trie has several levels, so every multiple of k fits, not only k .

Q31. [MCQ]

A fixed-stride trie reads k bits of the destination address at each step of a search. The prefix database holds prefixes of many different lengths. Some of these lengths are not multiples of k .

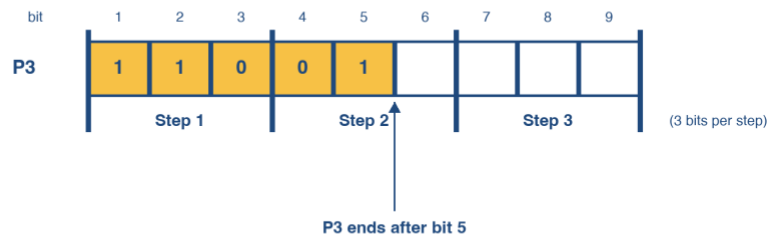
How is a prefix whose length is not a multiple of k stored in the trie?

- A. It is padded with 0 bits up to the next multiple of k , which makes one longer prefix.
- B. It is replaced by longer prefixes, at the next multiple of k , that cover the same addresses.
- C. It is cut back to the previous multiple of k , and the bits beyond that point are dropped.
- D. It is kept in a separate table, which is searched when the trie itself finds no match.

Correct answer: **B**

Why: Controlled prefix expansion replaces the prefix with longer prefixes, whose length is the next multiple of k . The new prefixes are all the ways to extend the original prefix to that length. Each one keeps the same next hop as the original. Together they cover exactly the same addresses as the original prefix. Padding with 0 bits covers only part of those addresses, and cutting back covers extra ones. A separate table would add a second search.

Q32. [MCQ]



Fixed-stride trie with a stride of 3

Figure: Fixed-stride trie with a stride of 3 (Module 5)

Consider a fixed-stride trie with a stride of 3. Each step of a search reads 3 bits of the address, so a search can stop only after 3, 6 or 9 bits, and so on. For this reason, every prefix stored in the trie must have a length of 3, 6, 9 and so on. The prefix database holds $P3 = 11001^*$, which is 5 bits long. $P3$ ends in the middle of a step, so the trie cannot store it as it is.

Which prefixes replace $P3$ in the trie?

- A. 110^* alone, cut back to the previous multiple of 3.
- B. 110010^* and 110011^* , the two prefixes of length 6.
- C. 110010^* alone, padding the missing bit with a 0.
- D. 110000^* to 110111^* , the eight prefixes under 110.

Correct answer: **B**

Why: Adding a 0 or a 1 to $P3$ gives 110010^* and 110011^* . Together they cover exactly the addresses of $P3$. Cutting $P3$ back to 110^* covers too many addresses. Adding only a 0 covers half of them. Expanding from 110 gives eight prefixes, and six of them cover addresses outside $P3$.

Q33. [MCQ]

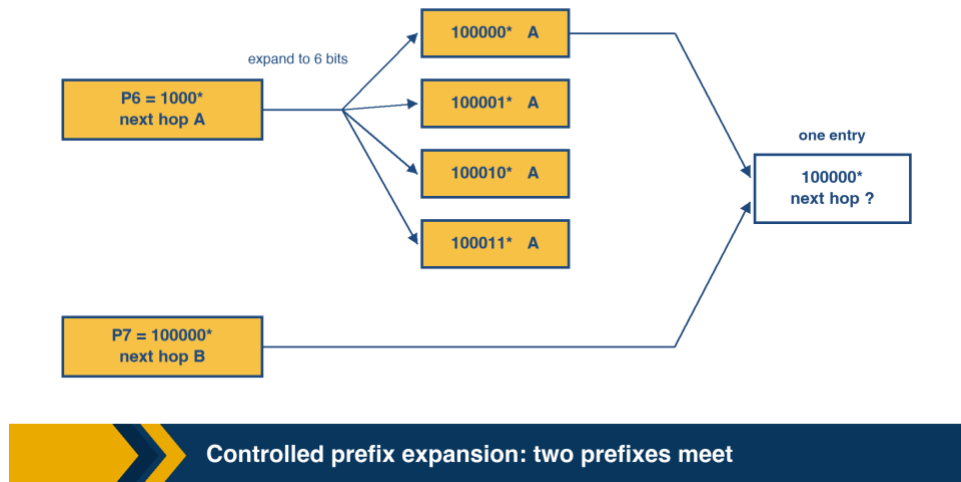


Figure: Controlled prefix expansion (Module 5)

Consider a router that extends every prefix to 6 bits using controlled prefix expansion. Its database holds two prefixes:

P6 = 1000*, next hop A

P7 = 100000*, next hop B

Expanding P6 gives 100000*, 100001*, 100010* and 100011*, all with next hop A. The first of these is the same as P7.

Which next hop does the router use for addresses that begin with 100000?

- A. Next hop A, because the expansion of P6 replaces the entry that P7 held.
- B. Next hop B, because P7 was longer than P6 before expansion, so it is the longer match.
- C. Both, because the router sends these packets alternately to A and to B.
- D. Neither, because two prefixes that collide cancel out and the entry is left empty.

Correct answer: **B**

Why: Before expansion, P7 had 6 bits and P6 had only 4. So P7 is the more specific prefix for these addresses, and the longest-match rule picks it. Expansion must not change that, so the router keeps P7 and drops the colliding expansion of P6.

Q34. [MCQ]

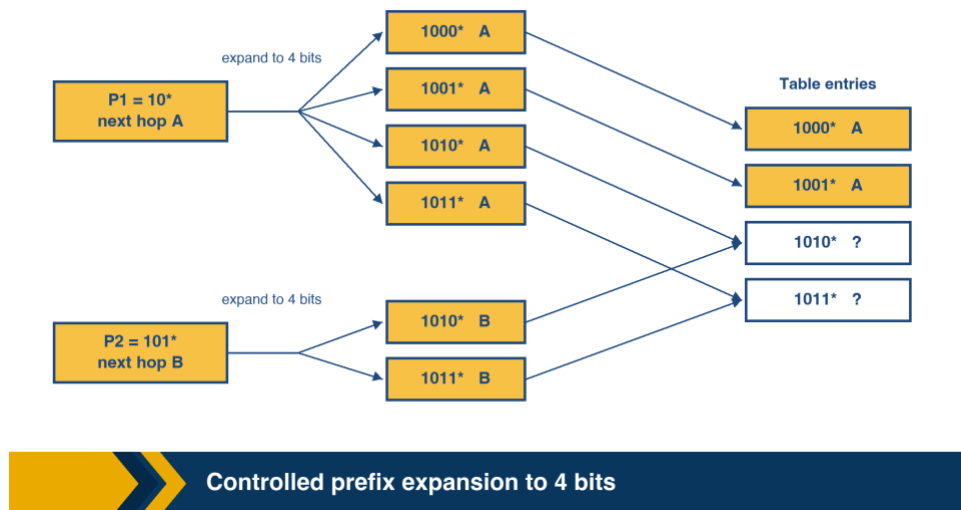


Figure: Controlled prefix expansion (Module 5)

Consider a router that extends every prefix to 4 bits using controlled prefix expansion. Its database holds two prefixes:

P1 = 10*, next hop A

P2 = 101*, next hop B

Expanding P1 gives 1000*, 1001*, 1010* and 1011*, all with next hop A. Expanding P2 gives 1010* and 1011*, both with next hop B.

Which next hop does the router store for 1010* and 1011*?

- A. Next hop A, because P1 was expanded first and its entries were already in place.
- B. Next hop B, because P2 was longer than P1 before expansion, so it is the longer match.
- C. Next hop A, because P1 covers more addresses, so its expansions take priority.
- D. Neither, because neither prefix was in the table at 4 bits before expansion.

Correct answer: **B**

Why: Before expansion, P2 had 3 bits and P1 had only 2. So P2 is the more specific prefix for addresses that begin with 101, and the longest-match rule picks it. Expansion must not change that, so the router stores next hop B for 1010 and 1011*. Next hop A stays for 1000* and 1001*.*

Multibit tries: Fixed-Stride

Q35. [MCQ]

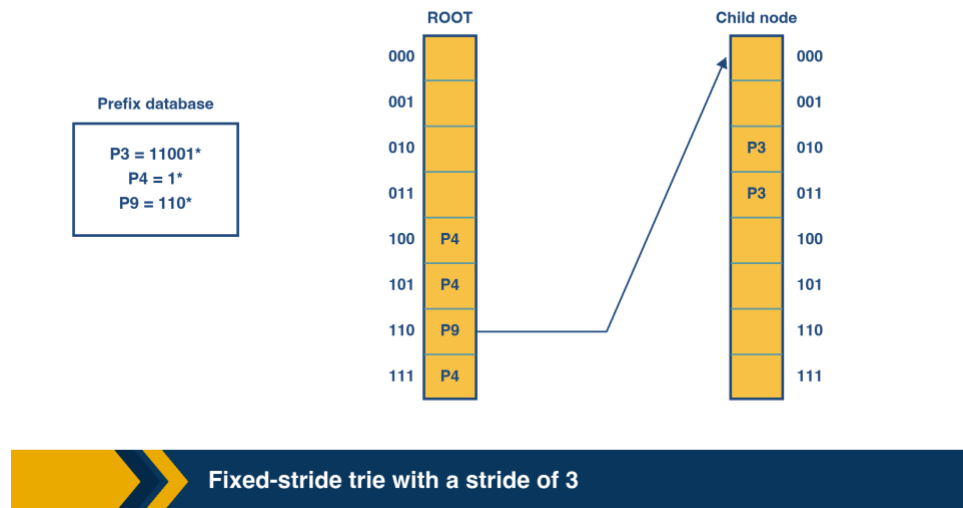


Figure: Fixed-stride trie with a stride of 3 (Module 5)

Consider a fixed-stride trie with a stride of 3 that stores three prefixes: P3 = 11001*, P4 = 1* and P9 = 110*. A destination address begins with 110000.

What does the search return?

- A. P9, the last stored prefix the search passed before it met the empty entry.
- B. No prefix: the empty entry ends the search with no match; the packet is dropped.
- C. P3, the last stored prefix the search passed before it met the empty entry.
- D. P4, the first stored prefix the search passed before it met the empty entry.

Correct answer: **A**

Why: The search remembers P9 at the 110 entry of the root. The 000 entry of the child node is empty, so the search stops and returns P9. An empty entry ends the search, but it does not cancel P9. P3 does not match 110000. P4 = 1* also matches, but P9 = 110* is longer.

Q36. [MCQ]

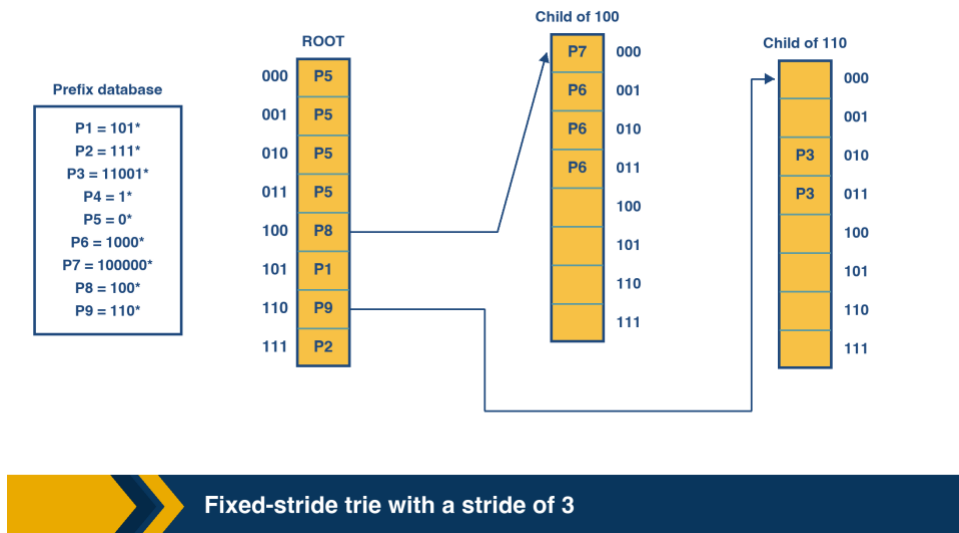


Figure: Fixed-stride trie with a stride of 3 (Module 5)

Consider a fixed-stride trie with a stride of 3. A destination address begins with 100011. At the root node, the search takes the 100 entry, which holds the stored prefix P8 and a pointer to a child node.

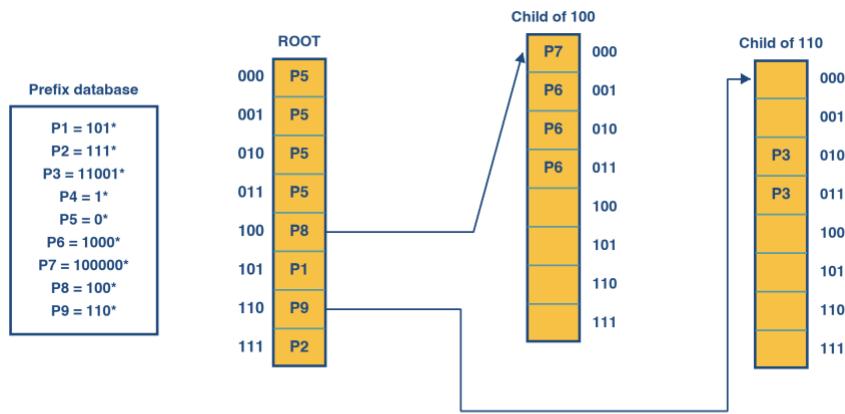
What does the search return?

- A. P8: the root's 100 entry holds P8, and a search returns the first stored prefix it finds.
- B. P6: the child's 011 entry holds P6 and has no pointer, so the search ends there.
- C. P7: the child node holds P7, the longest prefix in that node, so the search returns it.
- D. No prefix: the original database holds no length-6 prefix that begins with 100011.

Correct answer: **B**

Why: The search remembers P8 and follows the pointer. In the child of 100, the 011 entry holds an expansion of P6 and has no pointer. So the search ends and returns P6, the last prefix it passed, not the first. P7 = 100000* does not match 100011. P6 = 1000* matches, because the address begins with 1000. The prefix does not need to be 6 bits long.

Q37. [MCQ]



Fixed-stride trie with a stride of 3

Figure: Fixed-stride trie with a stride of 3 (Module 5)

Consider a fixed-stride trie with a stride of 3 that stores the prefixes P1 to P9.

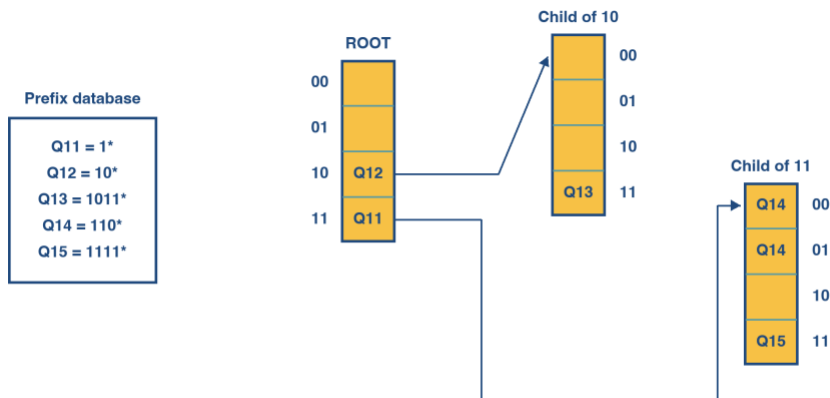
What does the 100 entry of the root node hold, and why?

- A. P8 and a pointer, because the pointer leads to the child node that holds the expansions of P8.
- B. P8 and a pointer, because P8 ends at this entry and the longer P6 and P7 continue below it.
- C. P8 and a pointer, because the pointer leads to the next hop of P8 and P8 picks the child to visit.
- D. P8 and a pointer, because a search that fails in the child node backs up to the root to read P8.

Correct answer: B

Why: $P8 = 100^*$ ends at the root's 100 entry, so the entry stores P8. P8 is already 3 bits long and has no expansions. $P6 = 1000^*$ and $P7 = 100000^*$ also begin with 100 and are longer. So the entry also needs a pointer to the child node that stores them. A search on 100 remembers P8 as it follows the pointer, and never backs up. It returns P8 only if the child node has no longer match.

Q38. [MCQ]



Multibit trie with a stride of 2

Figure: Multibit trie with a stride of 2 (Module 5)

Consider a multibit trie with a stride of 2, so each node reads 2 bits of the address. The trie was built from the following five prefixes, after controlled prefix expansion to lengths 2 and 4:

Q11 = 1*

Q12 = 10*

Q13 = 1011*

Q14 = 110*

Q15 = 1111*

A packet arrives with a destination address that begins with 1110.

Which prefix does the search return for this address?

- A. Q11 = 1*, the last prefix the search passed, stored in the 11 entry of the root.
- B. Q15 = 1111*, the last prefix the search passed, stored in the 11 entry of the child.
- C. No prefix: the 10 entry of the child node is empty, so the search finds no match.
- D. Q14 = 110*, the last prefix the search passed, stored in the child node below 11.

Correct answer: **A**

Why: At the root, the first 2 bits are 11. The 11 entry holds Q11, expanded from 1 to 11*, and a pointer. The search remembers Q11 and follows the pointer. In the child node, the 10 entry is empty. So the search stops and returns Q11, the prefix it remembered at the root. Q15 = 1111* does not match, because bits 3 and 4 are 10. Q14 = 110* expands to 1100* and 1101*, and neither matches.*

Multibit Tries: Variable Stride

Q39. [TF]

A variable-stride trie lets each node read a different number of bits. A dynamic program chooses these strides. It builds a unibit trie of the database, then works up from the bottom, reusing the best strides found for each subtrie. It runs about once a day.

The dynamic program chooses the strides to minimize the memory that the trie uses.

- True
- False

Correct answer: **True**

Why: The line rate and the memory speed fix the number of memory accesses first, which fixes the height of the trie. For that height, the dynamic program picks the strides that use the least memory.